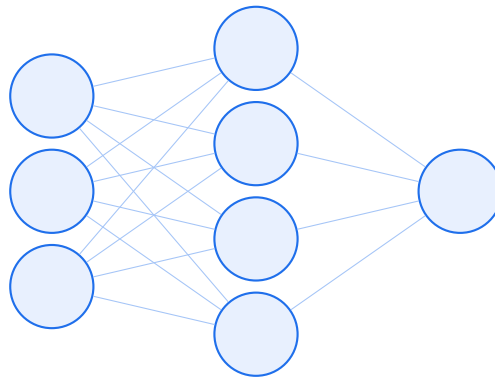

Neural Networks from Zero

A build-it-yourself textbook

from the single neuron to backpropagation,
derived, proved, and wired to working C++



Written to accompany Michael Nielsen's
Neural Networks and Deep Learning

and your `CNeural` project

For Pratham

so that every line is yours

Contents

1	The Neuron: One Number That Makes a Decision	1
1.1	Why we start here	1
1.2	The artificial neuron, defined	1
1.2.1	Why a weighted sum? Why a bias?	2
1.3	The activation function: from a hard switch to a smooth dial	2
1.3.1	The sigmoid	3
1.3.2	The derivative of the sigmoid (your first real derivation)	3
1.4	What this chapter bought you	5
2	Networks and the Cost: Turning “Wrong” Into One Number	7
2.1	From one neuron to a layer to a network	7
2.1.1	Why XOR needs the hidden layer	8
2.2	Measuring error: the cost function	8
2.2.1	The quadratic (MSE) cost	8
2.3	Learning as minimization	9
3	Gradient Descent: Walking Downhill on the Cost	11
3.1	The one-variable intuition	11
3.1.1	Why this provably decreases the cost (for small enough η)	11
3.2	Many variables: the gradient	12
3.3	Why we need matrices: the same arithmetic, batched	12
3.3.1	The matrix product, demystified	13
3.3.2	Two more operations you’ll need, and what makes them different	14
3.4	Where we are	14
4	Backpropagation: Computing Every Gradient at Once	15
4.1	The problem, stated plainly	15
4.1.1	Notation (so the proofs stay clean)	15
4.2	The key quantity: the error δ	15
4.3	BP1: the error at the output layer	16
4.4	BP2: pushing the error back one layer	17
4.5	BP3: the bias gradient	18

4.6	BP4: the weight gradient	19
4.7	The four equations together	20
4.8	The algorithm, and your training loop	20
4.9	A fully worked backward pass by hand	21
4.10	Why this is fast (the payoff)	22
4.11	What you now own	22

Chapter 1

The Neuron: One Number That Makes a Decision

1.1 Why we start here

You have a folder of C++ that trains on XOR and the loss goes down. It works. But you said it feels *copied* — like the code knows something you don't. That feeling is correct, and it is fixable. It is fixable because there is almost nothing in that folder that you cannot derive yourself with school calculus and patience. By the end of this book every line in `network.hpp` will feel like a sentence you wrote, not one you transcribed.

We start at level zero: a single neuron. Not a metaphor about brains — a precise piece of arithmetic that takes some numbers in and produces one number out.

Intuition first

A neuron is a tiny decision-maker. It listens to several inputs, decides how much it cares about each one, adds up the evidence, and then squashes the total into a clean “yes-ish / no-ish” answer between 0 and 1. That’s the whole story. Everything else — layers, backprop, MNIST — is just many of these wired together and tuned.

1.2 The artificial neuron, defined

Definition 1.1 (Artificial neuron). A neuron with n inputs is a function $\mathbb{R}^n \rightarrow \mathbb{R}$. It holds n *weights* $w_1, \dots, w_n$ and one *bias* b . Given inputs $x_1, \dots, x_n$ it computes a weighted sum

$$z = \sum_{k=1}^n w_k x_k + b,$$

and then applies an *activation function* σ to get its output

$$a = \sigma(z).$$

Two quantities, two names you will see forever. z is the **weighted input** (or pre-activation). a is the **activation** (the output). Hold onto these letters — they are exactly the `z` and `a` fields in your `Layer` struct.

In your code

In `network.hpp` your `Layer` stores `Matrix input`, `z`, `a`; and `forward` computes

```
z = weights * input + bias;  a = z.apply(sigmoid);
```

That is exactly $z = Wx + b$ then $a = \sigma(z)$, but for many neurons at once. We'll earn the

matrix form in Chapter 3.

1.2.1 Why a weighted sum? Why a bias?

Intuition first

The weights are *importances*. A big positive w_k means “input k pushes me toward firing.” A negative w_k means “input k pushes me toward staying quiet.” A weight near zero means “I don’t care about input k .” The bias is the neuron’s *eagerness*: how much it leans toward firing before it has heard any evidence at all.

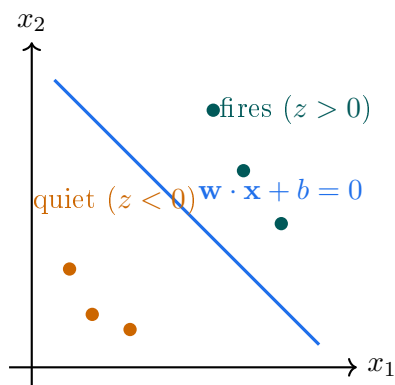
Make this concrete. Suppose you decide whether to go to the gym based on two inputs: $x_1 = 1$ if you slept well, $x_2 = 1$ if a friend is going. You weight sleep heavily ($w_1 = 3$) and the friend mildly ($w_2 = 1$). Your bias is $b = -2$ because by default you’re tired. Then

$$z = 3x_1 + 1x_2 - 2.$$

Slept well, friend going: $z = 3 + 1 - 2 = 2$ (positive — you go). Slept badly, no friend: $z = -2$ (negative — you stay). The geometry is a line $z = 0$ splitting the plane into “go” and “stay.” The weights tilt the line; the bias slides it.

Key idea

A single neuron draws one straight boundary (a hyperplane $\mathbf{w} \cdot \mathbf{x} + b = 0$) through input space. Points on one side fire; points on the other don’t. Learning means *moving that boundary* to fit the data, by adjusting $\mathbf{w}$ and b .



1.3 The activation function: from a hard switch to a smooth dial

The weighted sum z is any real number. We want an *output* that behaves like a decision. The crudest choice is a hard step:

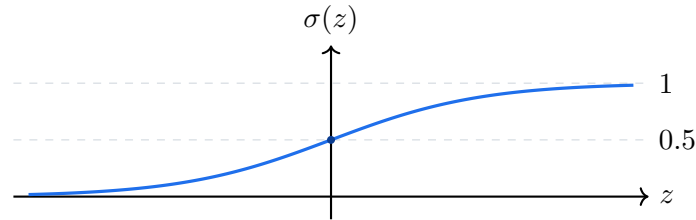
$$\text{step}(z) = \begin{cases} 1 & z \geq 0 \\ 0 & z < 0. \end{cases}$$

This is a *perceptron*. It works, but it has a fatal flaw for learning: it is flat everywhere and has a cliff at $z = 0$. Flat means “nudging a weight a little changes nothing,” and a cliff means “nudging it a tiny bit flips the answer completely.” Learning needs to feel the effect of small nudges. We need a smooth ramp instead of a cliff.

1.3.1 The sigmoid

Definition 1.2 (Sigmoid).

$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$



Read the picture. When z is very negative, e^{-z} is huge, so $\sigma \approx 0$. When z is very positive, $e^{-z} \approx 0$, so $\sigma \approx 1$. At $z = 0$, $\sigma = \frac{1}{1+1} = \frac{1}{2}$. So σ takes any real number and gently squashes it into $(0, 1)$ — a soft “probability of firing.”

Intuition first

The sigmoid is the step function with the cliff sanded down into a ramp. Far from the boundary it still says a confident 0 or 1. Near the boundary it is *uncertain*, and — crucially — it *slopes*. That slope is what lets the network feel “if I increase this weight a hair, my output rises by this much,” which is the entire mechanism of learning.

In your code

```
Scalar sigmoid(Scalar x){ return 1.0 / (1.0 + std::exp(-x)); }
```

A one-to-one transcription of the definition. Notice it is a `Scalar` $\rightarrow$ `Scalar` function; in your `Matrix::apply` you hand it to a whole matrix element-by-element. We meet `apply` properly in Chapter 3.

1.3.2 The derivative of the sigmoid (your first real derivation)

We will need $\sigma'(z)$ constantly, because “slope” is the whole point. The beautiful fact is that σ' can be written in terms of σ itself. Let us prove it step by step — nothing skipped.

Theorem 1.1 (Sigmoid derivative).

$$\sigma'(z) = \sigma(z) (1 - \sigma(z)).$$

Proof. Write $\sigma(z) = (1 + e^{-z})^{-1}$. Use the chain rule on the outer power and the inner exponential.

Step 1. Let $u = 1 + e^{-z}$, so $\sigma = u^{-1}$. Then

$$\frac{d\sigma}{dz} = \frac{d(u^{-1})}{du} \cdot \frac{du}{dz} = (-u^{-2}) \cdot \frac{du}{dz}.$$

Step 2. Differentiate the inside: $\frac{du}{dz} = \frac{d}{dz}(1 + e^{-z}) = -e^{-z}$ (the 1 dies, and e^{-z} brings down a -1).

Step 3. Substitute:

$$\sigma'(z) = -u^{-2} \cdot (-e^{-z}) = \frac{e^{-z}}{(1 + e^{-z})^2}.$$

Step 4. Now the trick — rewrite this in terms of σ . Split the fraction deliberately:

$$\frac{e^{-z}}{(1 + e^{-z})^2} = \underbrace{\frac{1}{1 + e^{-z}}}_{=\sigma(z)} \cdot \underbrace{\frac{e^{-z}}{1 + e^{-z}}}_{=?}.$$

The first factor is exactly $\sigma(z)$. For the second factor, add and subtract 1 in the numerator:

$$\frac{e^{-z}}{1 + e^{-z}} = \frac{(1 + e^{-z}) - 1}{1 + e^{-z}} = 1 - \frac{1}{1 + e^{-z}} = 1 - \sigma(z).$$

Step 5. Multiply the two factors:

$$\sigma'(z) = \sigma(z) (1 - \sigma(z)).$$

□

Key idea

$\sigma' = \sigma(1 - \sigma)$ is not a coincidence to memorize — it is why sigmoid networks are cheap to train. During the forward pass you already computed $a = \sigma(z)$. To get the slope you need for learning, you do *not* recompute exponentials; you just form $a(1 - a)$. The forward pass pays for the backward pass.

Watch out

Look at the shape of $\sigma' = \sigma(1 - \sigma)$. It is largest at $\sigma = \frac{1}{2}$ (value $\frac{1}{4}$) and shrinks to 0 as $\sigma \rightarrow 0$ or $\sigma \rightarrow 1$. So a neuron that is *very confident* (output near 0 or 1) has almost zero slope — it learns almost nothing. This is called **saturation**, and it is the single most important practical fact in this whole book. Keep it in your pocket; it explains slow training, why initialization matters, and half of the tricks in later chapters.

In your code

```
Scalar sigmoid_derivative(Scalar x){ x = sigmoid(x); return x * (1 - x); }
```

This takes the *pre-activation* z , computes $\sigma(z)$ into x , then returns $x(1-x) = \sigma(z)(1-\sigma(z))$. That is precisely the theorem you just proved. You now own this line.

Example 1.1 (Computing a neuron by hand). Inputs $\mathbf{x} = (1, 0.5, -0.5)$, weights $\mathbf{w} = (0.2, -0.4, 0.1)$, bias $b = 0.3$.

$$\begin{aligned} z &= (0.2)(1) + (-0.4)(0.5) + (0.1)(-0.5) + 0.3 \\ &= 0.2 - 0.2 - 0.05 + 0.3 = 0.25. \end{aligned}$$

$$a = \sigma(0.25) = \frac{1}{1 + e^{-0.25}} \approx \frac{1}{1 + 0.7788} \approx 0.562.$$

$$\sigma'(z) = a(1 - a) \approx 0.562 \times 0.438 \approx 0.246.$$

The neuron leans slightly toward firing (0.562) and is near its most-sensitive region (slope ≈ 0.25 , close to the max of 0.25). A good place to be while learning.

1.4 What this chapter bought you

You can now state, without looking anything up: a neuron is $a = \sigma(\mathbf{w} \cdot \mathbf{x} + b)$; the weights are importances and the bias is eagerness; the sigmoid is a smooth switch; its slope is $a(1-a)$; and confident neurons barely learn. Three lines of your `network.hpp` — `sigmoid`, `sigmoid_derivative`, and the body of `forward` — are now derivations you produced, not text you copied.

Next we wire neurons together and ask the question that makes learning possible: *how wrong are we, as one number?*

Chapter 2

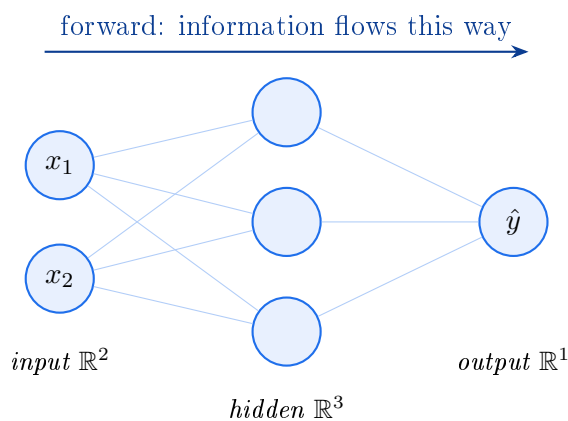
Networks and the Cost: Turning “Wrong” Into One Number

2.1 From one neuron to a layer to a network

One neuron draws one line. That is not enough — XOR (which your code solves) cannot be split by any single straight line, as we’ll prove in a moment. The fix is to stack neurons.

Definition 2.1 (Layer). A *layer* is a group of neurons that all read the same input vector and each produce one output. If the input has n numbers and the layer has m neurons, the layer is a function $\mathbb{R}^n \rightarrow \mathbb{R}^m$.

Definition 2.2 (Feedforward network). A *feedforward network* is a chain of layers where each layer’s output is the next layer’s input. The first layer reads the data; the last layer’s output is the prediction. “Feedforward” means information flows one way, input to output, with no loops.



In your code

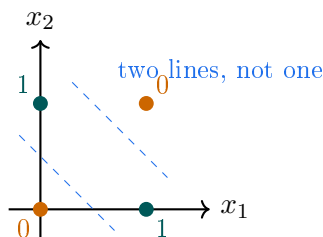
This is literally your XOR network: `net.add_layer(2, 3);` then `net.add_layer(3, 1);`. Two inputs, a hidden layer of three neurons, one output. Your `Network::forward` loops the layers, feeding each one’s output into the next:

```
for (auto &layer : layers) current = layer.forward(current);
```

2.1.1 Why XOR needs the hidden layer

Intuition first

XOR wants output 1 when the two inputs *differ* and 0 when they *match*. Plot the four points: $(0,0) \rightarrow 0$, $(1,1) \rightarrow 0$ sit on one diagonal; $(0,1) \rightarrow 1$, $(1,0) \rightarrow 1$ sit on the other. No single straight line can put both “1” corners on one side and both “0” corners on the other — they’re diagonally opposite. You need to bend the boundary, and bending requires a hidden layer.



The hidden layer learns two boundaries; the output neuron combines them. That is why `add_layer(2,3)` exists in your code and a single layer would never converge on XOR.

2.2 Measuring error: the cost function

To *learn*, the network needs to know how wrong it is — and it needs that wrongness as a *single number* it can try to shrink. That number is the **cost** (also: loss, objective).

Intuition first

Think of cost as a score in golf: lower is better, zero is perfect. We want one knob-readable number that is large when predictions are far from targets and zero when they match exactly. Then “learning” becomes the concrete task: *turn the weights and biases so this number goes down*.

2.2.1 The quadratic (MSE) cost

Definition 2.3 (Quadratic cost, one example). For a prediction vector $\mathbf{a} = (a_1, \dots, a_m)$ and target $\mathbf{y} = (y_1, \dots, y_m)$,

$$C = \frac{1}{2} \sum_{j=1}^m (a_j - y_j)^2 = \frac{1}{2} \|\mathbf{a} - \mathbf{y}\|^2.$$

Three design choices, each with a reason:

Why squared? The difference $a_j - y_j$ can be positive or negative; squaring makes every mismatch count positively (so errors can’t cancel) and punishes big misses far more than small ones. A miss of 0.4 contributes 0.16; a miss of 0.8 contributes 0.64 — four times as much for twice the error. The network is pushed hardest to fix its worst predictions.

Why sum over outputs? One number for the whole prediction, not one per output neuron.

Why the $\frac{1}{2}$? Pure convenience that you will thank yourself for. When we differentiate $(a_j - y_j)^2$, the chain rule drops a factor of 2 out front: $\frac{d}{da_j}(a_j - y_j)^2 = 2(a_j - y_j)$. The $\frac{1}{2}$ cancels that 2,

leaving the clean $(a_j - y_j)$. It changes the scale of the cost but not the location of its minimum, so it is free to include.

In your code

```

1 Scalar mse_loss(const Matrix &prediction, const Matrix &target)
  {
2   Matrix error = prediction - target;           // a_j - y_j
3   error = error.apply([](Scalar x){ return x*x; }); // (a_j -
      y_j)^2
4   Scalar sum = 0;
5   for (Scalar it : error.matrix) sum += it;     // sum over j
6   sum *= 0.5;                                   // the 1/2
7   return sum;
8 }

```

Line for line: subtract to get the errors, square them, sum them, halve. This is $\frac{1}{2} \sum_j (a_j - y_j)^2$. You wrote the definition; now you know why each piece is there.

Example 2.1 (Cost by hand). Prediction $\mathbf{a} = (0.5, 0.5)$, target $\mathbf{y} = (1.0, 0.0)$.

$$C = \frac{1}{2} [(0.5 - 1.0)^2 + (0.5 - 0.0)^2] = \frac{1}{2} [0.25 + 0.25] = 0.25.$$

Matches the value your `mse_loss` test prints. Halve the sum of squared misses — that's the whole computation.

2.3 Learning as minimization

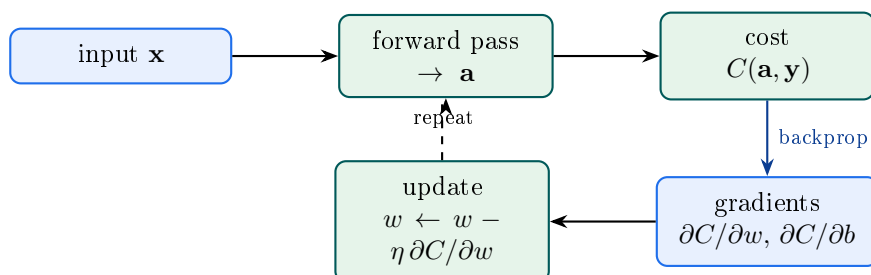
Now the problem has a crisp shape. The cost C depends on the data (fixed) and on every weight and bias in the network (adjustable). So C is a function of the parameters:

$$C = C(w_1, w_2, \dots, b_1, b_2, \dots).$$

Learning is finding the parameter values that make C as small as possible. That is an optimization problem, and the tool we use to solve it is gradient descent — the subject of the next chapter.

Key idea

Everything so far has been setup for one sentence: *train the network* = minimize C over the weights and biases. The forward pass computes C . The backward pass (backprop) computes how to change each parameter to lower C . Two halves of one loop.



You have now built the whole skeleton: neurons, layers, a network, and a single number that says how wrong it is. The rest of the book is about that downward arrow — how to compute it exactly and cheaply.

Chapter 3

Gradient Descent: Walking Downhill on the Cost

3.1 The one-variable intuition

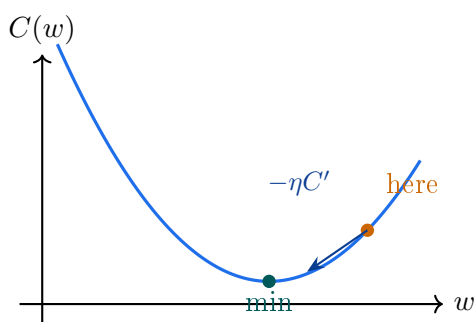
Intuition first

Imagine the cost as a hilly landscape and you are standing on it blindfolded, wanting to reach the lowest point. You can't see the valley, but you *can* feel the slope under your feet. The strategy is dumb and unbeatable: feel which way is downhill, take a small step that way, repeat. The slope is the derivative. "Take a step downhill" is gradient descent.

Start with a function of one variable, $C(w)$. The derivative $C'(w)$ is the slope. If $C'(w) > 0$ the function rises as w increases, so to *decrease* C we should *decrease* w . If $C'(w) < 0$ it's the reverse. Both cases are captured by one rule:

$$w \leftarrow w - \eta C'(w),$$

where $\eta > 0$ is the **learning rate** (step size). Check the signs: if $C' > 0$ we subtract a positive amount (move left, downhill); if $C' < 0$ we subtract a negative amount (move right, downhill). The minus sign *is* "go downhill."



3.1.1 Why this provably decreases the cost (for small enough η)

Theorem 3.1 (Descent). Let C be differentiable. Take the step $\Delta w = -\eta C'(w)$. Then for small enough $\eta > 0$, $C(w + \Delta w) \leq C(w)$, with equality only when $C'(w) = 0$.

Proof. The first-order Taylor approximation of C around w is

$$C(w + \Delta w) \approx C(w) + C'(w) \Delta w.$$

Substitute $\Delta w = -\eta C'(w)$:

$$C(w + \Delta w) \approx C(w) + C'(w)(-\eta C'(w)) = C(w) - \eta (C'(w))^2.$$

The term $\eta(C'(w))^2$ is ≥ 0 because it is a positive number times a square. So we subtract something non-negative from $C(w)$: the cost goes down (or stays put, only if $C'(w) = 0$, i.e. we are already at a flat point). The approximation is good precisely when Δw is small, which is why η must not be too large. $\square$

Watch out

The proof relies on the step being *small*. If η is too big, the Taylor approximation breaks and you can overshoot the valley — the cost can bounce or diverge. If η is too small, you crawl. Choosing η is a craft; your XOR code uses $\eta = 0.5$, which is aggressive but works because the problem is tiny.

3.2 Many variables: the gradient

A real network has thousands of weights. The cost is $C(w_1, \dots, w_N, b_1, \dots)$. The generalization of “slope” to many variables is the **gradient**: the vector of all partial derivatives.

Definition 3.1 (Gradient).

$$\nabla C = \left(\frac{\partial C}{\partial w_1}, \frac{\partial C}{\partial w_2}, \dots, \frac{\partial C}{\partial b_1}, \dots \right).$$

Each partial derivative $\partial C / \partial w_k$ answers: “if I nudge *only* w_k and freeze everything else, how does C change?”

Key idea

The gradient points in the direction of *steepest ascent* — the fastest way uphill. So $-\nabla C$ is steepest descent. The update rule for every parameter at once is

$$\theta \leftarrow \theta - \eta \nabla C,$$

i.e. for each weight $w_k \leftarrow w_k - \eta \partial C / \partial w_k$ and each bias $b_j \leftarrow b_j - \eta \partial C / \partial b_j$. This single line is how every neural network on earth learns. Backpropagation (next chapter) is simply the efficient way to compute all those partial derivatives.

In your code

```
weights = weights - weight_grad.scalar_multiply(lr);
```

That is $w \leftarrow w - \eta \partial C / \partial w$ in matrix form: `weight_grad` is the gradient, `lr` is η , `scalar_multiply` scales the gradient, and the `operator-` subtracts it. The bias line is the same with b . The only mystery left is how `weight_grad` gets computed — and that is exactly what Chapter 4 derives.

3.3 Why we need matrices: the same arithmetic, batched

A layer applies the *same* operation — weighted sum then activation — to many neurons. Writing per-neuron loops is correct but unreadable. Linear algebra packages all of it into three symbols.

Definition 3.2 (Layer in matrix form). Stack a layer’s weights into a matrix W (one row per neuron, one column per input) and biases into a column vector $\mathbf{b}$. For input column vector $\mathbf{x}$,

$$\mathbf{z} = W\mathbf{x} + \mathbf{b}, \quad \mathbf{a} = \sigma(\mathbf{z}),$$

where σ is applied element-wise.

Unpack the dimensions, because they are where bugs hide and where understanding lives. If the layer has m neurons and n inputs:

- W is $m \times n$ — row i holds the n weights of neuron i .
- $\mathbf{x}$ is $n \times 1$.
- $W\mathbf{x}$ is $m \times 1$: row i is $\sum_k W_{ik}x_k$, exactly neuron i ’s weighted sum.
- $\mathbf{b}$ is $m \times 1$, added element-wise.
- $\mathbf{a} = \sigma(\mathbf{z})$ is $m \times 1$: the layer’s outputs.

$$\underbrace{\begin{bmatrix} W_{11} & W_{12} & W_{13} \\ W_{21} & W_{22} & W_{23} \end{bmatrix}}_{m \times n = 2 \times 3} \underbrace{\begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}}_{n \times 1} + \underbrace{\begin{bmatrix} b_1 \\ b_2 \end{bmatrix}}_{m \times 1} = \underbrace{\begin{bmatrix} z_1 \\ z_2 \end{bmatrix}}_{m \times 1}$$

In your code

Your `Layer` constructor builds exactly these shapes:

```
weights(outputs, inputs, true), bias(outputs, 1, true)
```

`outputs`= m rows, `inputs`= n columns for the weights; `outputs` $\times 1$ for the bias. And `forward` is

```
z = weights * input + bias; a = z.apply(sigmoid);
```

$W\mathbf{x} + \mathbf{b}$ then $\sigma(\mathbf{z})$, element for element. Your `operator*` does the matrix product, `operator+` the bias add, `apply` the element-wise sigmoid.

3.3.1 The matrix product, demystified

The one operation worth slowing down on is matrix multiplication, because it is the forward pass. The rule: entry (i, j) of AB is the dot product of row i of A with column j of B :

$$(AB)_{ij} = \sum_k A_{ik}B_{kj}.$$

This is only defined when A ’s column count equals B ’s row count (the “inner” dimensions match).

Example 3.1 (Matrix multiply by hand).

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \begin{bmatrix} 7 & 8 \\ 9 & 10 \\ 11 & 12 \end{bmatrix} = \begin{bmatrix} 1 \cdot 7 + 2 \cdot 9 + 3 \cdot 11 & 1 \cdot 8 + 2 \cdot 10 + 3 \cdot 12 \\ 4 \cdot 7 + 5 \cdot 9 + 6 \cdot 11 & 4 \cdot 8 + 5 \cdot 10 + 6 \cdot 12 \end{bmatrix} = \begin{bmatrix} 58 & 64 \\ 139 & 154 \end{bmatrix}.$$

$(2 \times 3)(3 \times 2) = (2 \times 2)$: outer dimensions survive, inner dimension (3) is summed away.

In your code

```

1 for (i64 i = 0; i < rows; ++i)
2   for (i64 j = 0; j < other.cols; ++j) {
3     Scalar sum = 0;
4     for (i64 k = 0; k < other.rows; ++k)
5       sum += at(i,k) * other.at(k,j);    // dot product, row i .
6       col j
7     result.at(i,j) = sum;
  }

```

The triple loop is the definition $\sum_k A_{ik}B_{kj}$ made literal. The check `if (cols != other.rows) throw` is the “inner dimensions must match” rule. The worked example above is the exact test case commented in your `main.cpp` — run it and you’ll get $\begin{smallmatrix} 58 & 64 \\ 139 & 154 \end{smallmatrix}$.

3.3.2 Two more operations you’ll need, and what makes them different

Transpose $A^\top$ flips rows and columns: $(A^\top)_{ij} = A_{ji}$. An $m \times n$ matrix becomes $n \times m$. We need it in backprop to send error *backwards* through a layer — a forward weight matrix W (shape $m \times n$) gets used as $W^\top$ (shape $n \times m$) to push signals the other way.

Hadamard product $A \odot B$ multiplies element by element: $(A \odot B)_{ij} = A_{ij}B_{ij}$. Both matrices must be the same shape, and the result is that shape. This is *not* matrix multiplication.

Watch out

Do not confuse the two products. Matrix multiply $W\mathbf{x}$ *mixes* inputs together (each output is a weighted blend of all inputs) — it’s how signals combine. Hadamard $\mathbf{a} \odot \mathbf{b}$ keeps each position separate — it’s how we apply a per-neuron scaling, like multiplying each error by that neuron’s own slope. In Chapter 4 you’ll see the rule of thumb: **matrix-multiply to move between layers, Hadamard to apply an activation’s slope within a layer.**

In your code

Your `transpose()` writes `result.at(j,i) = at(i,j)` — the index swap, exactly. Your `hadamard()` multiplies `at(i,j) * other.at(i,j)` with a same-shape check. Both are one-liners that mirror the definitions. You now own the entire `Matrix` class — every method is a definition you can re-derive.

3.4 Where we are

You can now read $\mathbf{z} = W\mathbf{x} + \mathbf{b}$, $\mathbf{a} = \sigma(\mathbf{z})$, and $\theta \leftarrow \theta - \eta \nabla C$ as plain statements, and you know why matrices show up (same arithmetic, batched) and which product does what. The single remaining gap is the gradient itself: $\partial C / \partial w$ and $\partial C / \partial b$ for every parameter, including the hidden ones buried behind other layers. Closing that gap is backpropagation. That is the heart of the book, and the next chapter.

Chapter 4

Backpropagation: Computing Every Gradient at Once

4.1 The problem, stated plainly

We need $\partial C/\partial w$ and $\partial C/\partial b$ for *every* weight and bias — including ones buried in hidden layers, far from the output where the error is measured. The naive approach (nudge one parameter, re-run the whole forward pass, see how C changed, repeat for all N parameters) costs N forward passes per step. For a real network that is hopeless.

Intuition first

Backprop is the insight that you don't need N separate experiments. The error at the output can be *routed backwards* through the network, and as it flows back, each parameter learns its own share of the blame in a single sweep. One forward pass to compute everything, one backward pass to assign credit. The chain rule is the routing machinery.

4.1.1 Notation (so the proofs stay clean)

We index layers by l . The last layer is L . For layer l :

- w_{jk}^l = weight *into* neuron j of layer l *from* neuron k of layer $l-1$.
- b_j^l, z_j^l, a_j^l = bias, weighted input, activation of neuron j in layer l .
- $z_j^l = \sum_k w_{jk}^l a_k^{l-1} + b_j^l$ and $a_j^l = \sigma(z_j^l)$.

Watch out

The index order w_{jk}^l (j = destination, k = source) looks backwards but is deliberate: it makes the forward pass a clean matrix product $\mathbf{z}^l = W^l \mathbf{a}^{l-1} + \mathbf{b}^l$ with no transpose. The price is a transpose appearing in the *backward* pass. This single convention choice is why $W^\top$ shows up later — it's bookkeeping, not deep magic.

4.2 The key quantity: the error δ

Instead of attacking $\partial C/\partial w$ directly, we introduce a helper that makes everything fall out cleanly.

Definition 4.1 (Error of a neuron). The error of neuron j in layer l is

$$\delta_j^l := \frac{\partial C}{\partial z_j^l}.$$

It measures how much the cost would change if you jiggled that neuron's *weighted input* z_j^L .

Intuition first

Why differentiate with respect to z and not the output a or the weights directly? Because z is the chokepoint. Every weight and the bias of a neuron flow *into* z , and z flows out to everything downstream. If you know how sensitive the cost is to z (that's δ), you can read off the sensitivity to the weights and bias with almost no extra work — they're one short chain-rule hop away. δ is the wholesale quantity; the weight and bias gradients are retail, derived from it.

We now prove four equations. Together they let you compute every δ , and from the δ 's every gradient. These are Nielsen's BP1–BP4.

4.3 BP1: the error at the output layer

Intuition first

At the very last layer, error is easy to assign because the output *is* what the cost looks at directly. Two things decide a output neuron's error: how wrong its output is (how far a_j^L is from the target), and how responsive its output is to its input (the slope $\sigma'(z_j^L)$). A neuron that's wrong but saturated — flat slope — can barely be moved, so its effective error is small. BP1 is exactly “wrongness $\times$ responsiveness.”

Theorem 4.1 (BP1).

$$\delta_j^L = \frac{\partial C}{\partial a_j^L} \sigma'(z_j^L).$$

In vector form, $\boldsymbol{\delta}^L = \nabla_a C \odot \sigma'(\mathbf{z}^L)$.

Proof. By definition $\delta_j^L = \partial C / \partial z_j^L$. The cost C depends on z_j^L *only through* the activation $a_j^L = \sigma(z_j^L)$ — changing z_j^L changes a_j^L , which changes C , and there is no other path. So the chain rule gives a single term:

$$\delta_j^L = \frac{\partial C}{\partial z_j^L} = \frac{\partial C}{\partial a_j^L} \frac{\partial a_j^L}{\partial z_j^L}.$$

The second factor is $\partial a_j^L / \partial z_j^L = \partial \sigma(z_j^L) / \partial z_j^L = \sigma'(z_j^L)$. Substituting,

$$\delta_j^L = \frac{\partial C}{\partial a_j^L} \sigma'(z_j^L). \quad \square$$

Making it concrete with our cost. For the quadratic cost $C = \frac{1}{2} \sum_j (a_j^L - y_j)^2$, the only term involving a_j^L is $\frac{1}{2} (a_j^L - y_j)^2$, so

$$\frac{\partial C}{\partial a_j^L} = \frac{1}{2} \cdot 2(a_j^L - y_j) = a_j^L - y_j.$$

(There's the $\frac{1}{2}$ paying off, exactly as promised in Chapter 2.) Therefore

$$\delta_j^L = (a_j^L - y_j) \sigma'(z_j^L)$$

In your code

```

1 Matrix output_delta =
2   (net.layers[1].a - targets[ex])
3   .hadamard(net.layers[1].z.apply(sigmoid_derivative));

```

Read it against the boxed formula. $(\mathbf{a} - \mathbf{y})$ is $a_j^L - y_j$ (the $\partial C / \partial a$ term for MSE). $\mathbf{z}.\text{apply}(\text{sigmoid_derivative})$ is $\sigma'(z_j^L)$. $\text{.hadamard}(\dots)$ multiplies them position by position — that's the $\odot$ in $\boldsymbol{\delta}^L = (\mathbf{a}^L - \mathbf{y}) \odot \sigma'(\mathbf{z}^L)$. This line is BP1, and you just proved every symbol in it.

4.4 BP2: pushing the error back one layer

Intuition first

A hidden neuron never sees the target. How can it be “wrong”? Through its consequences. Neuron k in layer l feeds every neuron in layer $l + 1$. Its blame is the blame of those downstream neurons, routed back through the weights that connect them — and then scaled by its own responsiveness $\sigma'(z_k^l)$. “How much error did I cause downstream, weighted by how strongly I’m connected to each downstream neuron, times how much I can actually move.”

Theorem 4.2 (BP2).

$$\delta_k^l = \left(\sum_j w_{jk}^{l+1} \delta_j^{l+1} \right) \sigma'(z_k^l).$$

In vector form, $\boldsymbol{\delta}^l = ((W^{l+1})^\top \boldsymbol{\delta}^{l+1}) \odot \sigma'(\mathbf{z}^l)$.

Proof. Start from the definition $\delta_k^l = \partial C / \partial z_k^l$. The weighted input z_k^l affects the cost only by feeding forward into the next layer’s weighted inputs z_j^{l+1} (for all j). Since there are several such downstream neurons, the multivariable chain rule sums over all of them:

$$\delta_k^l = \frac{\partial C}{\partial z_k^l} = \sum_j \frac{\partial C}{\partial z_j^{l+1}} \frac{\partial z_j^{l+1}}{\partial z_k^l} = \sum_j \delta_j^{l+1} \frac{\partial z_j^{l+1}}{\partial z_k^l}.$$

(We used $\partial C / \partial z_j^{l+1} = \delta_j^{l+1}$ — the definition of error one layer up. This is the recursion: δ^l in terms of δ^{l+1} .)

Now compute the local factor $\partial z_j^{l+1} / \partial z_k^l$. By the forward rule,

$$z_j^{l+1} = \sum_{k'} w_{jk'}^{l+1} a_{k'}^l + b_j^{l+1} = \sum_{k'} w_{jk'}^{l+1} \sigma(z_{k'}^l) + b_j^{l+1}.$$

Differentiate with respect to z_k^l . Only the $k' = k$ term in the sum depends on z_k^l ; the rest vanish:

$$\frac{\partial z_j^{l+1}}{\partial z_k^l} = w_{jk}^{l+1} \sigma'(z_k^l).$$

Substitute back:

$$\delta_k^l = \sum_j \delta_j^{l+1} w_{jk}^{l+1} \sigma'(z_k^l) = \left(\sum_j w_{jk}^{l+1} \delta_j^{l+1} \right) \sigma'(z_k^l).$$

The $\sigma'(z_k^l)$ factored out because it doesn’t depend on j . □ □

Why the transpose. The sum $\sum_j w_{jk}^{l+1} \delta_j^{l+1}$ has the summation index j on the *first* (row) slot of w . Ordinary matrix-vector multiply $W\delta$ sums over the *second* (column) slot. To match, we flip W : using $(W^{l+1})^\top$ puts k on the rows and j on the columns, so $(W^{l+1})^\top \delta^{l+1}$ computes exactly $\sum_j w_{jk}^{l+1} \delta_j^{l+1}$ for each k . That is the entire reason a transpose appears in backprop — it re-routes the weights to carry signal backwards.

In your code

```
1 Matrix hidden_delta =
2     (net.layers[1].weights.transpose() * output_delta)
3     .hadamard(net.layers[0].z.apply(sigmoid_derivative));
```

`layers[1].weights.transpose()` is $(W^{l+1})^\top$. Multiplying by `output_delta` (δ^{l+1}) does $\sum_j w_{jk}^{l+1} \delta_j^{l+1}$. `.hadamard(layers[0].z.apply(sigmoid_derivative))` multiplies by $\sigma'(z^l)$. Line = BP2, symbol for symbol. The transpose you maybe copied uneasily is now a thing you derived from a chain-rule index mismatch.

4.5 BP3: the bias gradient

Intuition first

The bias feeds straight into z with a coefficient of exactly 1 — nudge the bias by ϵ and z moves by ϵ , no scaling. So the sensitivity of the cost to the bias is identical to its sensitivity to z , which we already named δ . The bias gradient is just the error. Nothing to compute.

Theorem 4.3 (BP3).

$$\frac{\partial C}{\partial b_j^l} = \delta_j^l.$$

Proof. C depends on b_j^l only through z_j^l , so the chain rule gives $\partial C / \partial b_j^l = (\partial C / \partial z_j^l)(\partial z_j^l / \partial b_j^l) = \delta_j^l \cdot (\partial z_j^l / \partial b_j^l)$. From $z_j^l = \sum_k w_{jk}^l a_k^{l-1} + b_j^l$, differentiating with respect to b_j^l kills every term except the bias, whose coefficient is 1: $\partial z_j^l / \partial b_j^l = 1$. Hence $\partial C / \partial b_j^l = \delta_j^l$. $\square$ $\square$

In your code

```
Matrix output_bias_grad = output_delta; Matrix hidden_bias_grad =
    hidden_delta;
```

A direct copy, no arithmetic — because BP3 says the bias gradient *is* the error. That “why are we just assigning it?” moment in your code is now answered: $\partial C / \partial b = \delta$.

4.6 BP4: the weight gradient

Intuition first

How much should a weight change? It depends on two things: how much error sits at the neuron the weight feeds into (δ_j^l), and how active the input coming through that weight was (a_k^{l-1}). A weight carrying a strong input into a high-error neuron is very responsible for the cost — big gradient. A weight carrying a near-zero input matters little no matter the error — small gradient. BP4 is “error at the destination $\times$ activation at the source.”

Theorem 4.4 (BP4).

$$\frac{\partial C}{\partial w_{jk}^l} = a_k^{l-1} \delta_j^l.$$

In matrix form, $\nabla_{W^l} C = \boldsymbol{\delta}^l (\mathbf{a}^{l-1})^\top$.

Proof. C depends on w_{jk}^l only through z_j^l (that weight feeds exactly one neuron’s weighted input). Chain rule:

$$\frac{\partial C}{\partial w_{jk}^l} = \frac{\partial C}{\partial z_j^l} \frac{\partial z_j^l}{\partial w_{jk}^l} = \delta_j^l \frac{\partial z_j^l}{\partial w_{jk}^l}.$$

From $z_j^l = \sum_{k'} w_{jk'}^l a_{k'}^{l-1} + b_j^l$, only the $k' = k$ term contains w_{jk}^l , and its derivative is the coefficient a_k^{l-1} :

$$\frac{\partial z_j^l}{\partial w_{jk}^l} = a_k^{l-1}.$$

Therefore $\partial C / \partial w_{jk}^l = a_k^{l-1} \delta_j^l = \delta_j^l a_k^{l-1}$. □ □

Why the outer product. We want a matrix the same shape as W^l ($m \times n$: m neurons, n inputs), with entry (j, k) equal to $\delta_j^l a_k^{l-1}$. That is exactly the outer product of the column vector $\boldsymbol{\delta}^l$ ($m \times 1$) with the row vector $(\mathbf{a}^{l-1})^\top$ ($1 \times n$): $\boldsymbol{\delta}^l (\mathbf{a}^{l-1})^\top$ has entry $(j, k) = \delta_j^l a_k^{l-1}$. Shapes: $(m \times 1)(1 \times n) = (m \times n)$, matching W^l .

In your code

```
Matrix output_weight_grad = output_delta * net.layers[1].input.transpose();
```

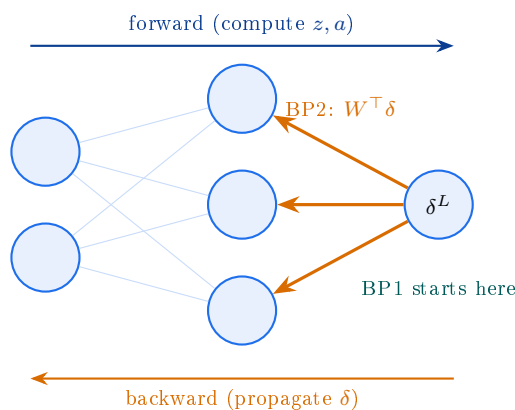
`output_delta` is $\boldsymbol{\delta}^l$ (a column). `layers[1].input` is $\mathbf{a}^{l-1}$, the activation that fed this layer — and `.transpose()` makes it the row $(\mathbf{a}^{l-1})^\top$. The `*` is the outer product $\boldsymbol{\delta}^l (\mathbf{a}^{l-1})^\top$. Result shape matches `weights`, so the later subtraction `weights - grad.scalar_multiply(lr)` type-checks and means $w \leftarrow w - \eta \partial C / \partial w$. BP4, owned.

4.7 The four equations together

Key idea

- (BP1) $\delta^L = \nabla_a C \odot \sigma'(\mathbf{z}^L)$ error at the output
- (BP2) $\delta^l = ((W^{l+1})^\top \delta^{l+1}) \odot \sigma'(\mathbf{z}^l)$ error pushed back a layer
- (BP3) $\frac{\partial C}{\partial b_j^l} = \delta_j^l$ bias gradient = error
- (BP4) $\frac{\partial C}{\partial w_{jk}^l} = a_k^{l-1} \delta_j^l$ weight gradient = source act. $\times$ dest. error

BP1 starts the fire at the output. BP2 carries it backward, layer by layer. BP3 and BP4 read off the gradients from the errors. That is all of backpropagation.



4.8 The algorithm, and your training loop

Putting it in order, one training example:

1. **Forward.** Compute $\mathbf{z}^l, \mathbf{a}^l$ for every layer, storing them.
2. **Output error (BP1).** $\delta^L = (\mathbf{a}^L - \mathbf{y}) \odot \sigma'(\mathbf{z}^L)$.
3. **Backpropagate (BP2).** For $l = L - 1, \dots$: $\delta^l = ((W^{l+1})^\top \delta^{l+1}) \odot \sigma'(\mathbf{z}^l)$.
4. **Gradients (BP3, BP4).** $\partial C / \partial \mathbf{b}^l = \delta^l$, $\nabla_{W^l} C = \delta^l (\mathbf{a}^{l-1})^\top$.
5. **Update.** $W^l \leftarrow W^l - \eta \nabla_{W^l} C$, $\mathbf{b}^l \leftarrow \mathbf{b}^l - \eta \delta^l$.

In your code

Your XOR loop is this algorithm, unrolled for two layers:

```
1 Matrix h_out = net.layers[0].forward(inputs[ex]); // forward (
   step 1)
2 Matrix o_out = net.layers[1].forward(h_out);
3 total_loss += mse_loss(o_out, targets[ex]);
```

```

4
5 Matrix output_delta = (net.layers[1].a - targets[ex])
6   .hadamard(net.layers[1].z.apply(sigmoid_derivative));
7   // BP1
8 Matrix hidden_delta = (net.layers[1].weights.transpose() *
9   output_delta)
10   .hadamard(net.layers[0].z.apply(sigmoid_derivative));
11   // BP2
12 Matrix output_bias_grad = output_delta;
13   // BP3
14 Matrix hidden_bias_grad = hidden_delta;
15 Matrix output_weight_grad = output_delta * net.layers[1].input.
16   transpose(); // BP4
17 Matrix hidden_weight_grad = hidden_delta * net.layers[0].input.
18   transpose();
19
20 net.layers[1].weights = net.layers[1].weights -
21   output_weight_grad.scalar_multiply(1r); // update
22 net.layers[1].bias = net.layers[1].bias - output_bias_grad
23   .scalar_multiply(1r);
24 net.layers[0].weights = net.layers[0].weights -
25   hidden_weight_grad.scalar_multiply(1r);
26 net.layers[0].bias = net.layers[0].bias - hidden_bias_grad
27   .scalar_multiply(1r);

```

Every line now has a name and a proof behind it. Nothing here is copied; it is the five-step algorithm written in your Matrix API.

4.9 A fully worked backward pass by hand

Numbers make it real. Take a tiny net: input $\rightarrow$ 1 hidden neuron $\rightarrow$ 1 output neuron (so every matrix is 1×1 and we can do it on paper). Suppose after the forward pass:

$$a^{l-1} = 0.6 \text{ (input act.)}, \quad z^L = 0.4, \quad a^L = \sigma(0.4) \approx 0.599, \quad y = 1.$$

BP1: $\sigma'(z^L) = a^L(1 - a^L) \approx 0.599 \times 0.401 \approx 0.240$.

$$\delta^L = (a^L - y) \sigma'(z^L) = (0.599 - 1)(0.240) = (-0.401)(0.240) \approx -0.0962.$$

BP3: $\partial C / \partial b^L = \delta^L \approx -0.0962$.

BP4: $\partial C / \partial w^L = a^{l-1} \delta^L = 0.6 \times (-0.0962) \approx -0.0577$.

Update with $\eta = 0.5$: $w^L \leftarrow w^L - 0.5(-0.0577) = w^L + 0.0289$. The weight *increases* — sensible, because our output 0.599 was below the target 1, so we strengthen the weight to push the output up. The math agrees with the intuition. That agreement, every time, is the sign you’ve understood it.

4.10 Why this is fast (the payoff)

Key idea

The naive method needed one forward pass per parameter — N passes. Backprop needs *one* forward pass plus *one* backward pass, total, to get *all* N gradients. The backward pass costs about the same as the forward pass. So you compute the entire gradient for roughly the price of two forward passes, independent of how many parameters you have. That efficiency is the only reason training large networks is possible at all — and it is nothing more than the chain rule, applied with the bookkeeping you just did by hand.

4.11 What you now own

You proved all four backprop equations from the chain rule, intuition-first, and mapped each to the exact line in your XOR trainer. The transpose, the Hadamard, the bias-gradient-is-just-delta assignment, the outer product for weights — none of it is copied anymore. You could delete `network.hpp` and rewrite it from these five steps. The loss goes down because of mathematics you can reconstruct on a blank page. That is ownership.

Next, when you take this to MNIST (Stage 3): the same four equations, unchanged. What changes is scale — mini-batches, better initialization to dodge saturation, and possibly swapping the quadratic cost for cross-entropy to kill the slowdown BP1 inherits from σ' . Those are refinements on a foundation you now hold completely.